

IICPC Summer Hackathon 2026

Welcome to the **IICPC Summer Hackathon 2026**. Running from **May 9th to June 10th, 2026**, this competition is designed for top-tier systems engineers, competitive programmers, and algorithmic thinkers. Submissions will open during the final week of the hackathon.

While participants have complete freedom to select their tech stack and utilize AI development tools, we strongly emphasize the necessity of a deliberate, well-reasoned architectural thought process behind every technical decision. At IICPC, we champion hardcore engineering excellence. This is not a standard "demo-to-win" hackathon; we expect high-performance code, system resilience, and a deep understanding of scale and distributed systems.

Note: You can form teams of at most 3 people / team for this hackathon.

The Challenge

Your objective is to architect and build a **Distributed Benchmarking and Hosting Platform** designed to evaluate contestant-submitted **trading infrastructure**.

The platform must allow contestants to **upload their core code**—such as a **simulated orderbook or matching engine**. Your system must then securely host this submission, **expose predefined API or WebSocket endpoints**, and dynamically **spawn a massive, distributed fleet of "trading bots."** These **bots will bombard** the contestant's system **with concurrent orders to simulate peak market volatility**. Finally, your **platform must capture** granular **telemetry to assess the submitted code on latency, throughput, and correctness**, streaming the results to a live, dynamic leaderboard.

Architectural Components & Requirements

Teams must demonstrate engineering mastery by building a highly concurrent, resilient, and decoupled system. Your solution must encompass the following core components:

Component	Technical Specifications & Expectations
Submission & Sandboxing Engine	A secure pipeline where contestants upload their binaries or source code (e.g., C++, Rust, Go). The platform must containerize and deploy these submissions in strictly isolated environments to prevent malicious code execution and ensure fair resource allocation (e.g., CPU pinning, strict memory limits).
Distributed Load Generator (Bot Fleet)	The engine of the platform. You must build a scalable traffic generation service capable of spawning thousands of distributed bots. These bots will simulate diverse market participants, sending high-velocity FIX, REST, or WebSocket requests (Limit Orders, Market Orders, Cancels) to the contestant's endpoints.
Telemetry & Validation Ingestor	A low-latency tracking system monitoring the interactions between the bots and the contestant's exchange. It must accurately measure: • Latency: Order acknowledgment time (p50, p90, and p99 latencies). • Throughput: Maximum transactions per second (TPS) handled before failure. • Correctness: Validation of price-time priority and fill accuracy.
Real-Time Leaderboard & Analytics	A frontend interface that streams live metrics from the ongoing stress tests, ranking contestants dynamically based on a composite score of speed, stability, and algorithmic accuracy.

Expected Deliverables

1. **Working Infrastructure Prototype:** A fully functional platform demonstrating the complete pipeline: *Code Upload* → *Containerized Deployment* → *Distributed Load Testing* → *Real-Time Scoring*.
2. **Architecture Blueprint:** A comprehensive system design document detailing your microservices, inter-service communication protocols (e.g., gRPC, Kafka/Redpanda for metrics), data stores (e.g., TimescaleDB, Redis), and isolation strategies.
3. **Infrastructure as Code (IaC):** Automated deployment scripts (e.g., Terraform, Kubernetes manifests, or Docker Swarm configurations) proving that your platform can be spun up, configured, and scaled horizontally in a modern cloud environment.